



Review of Combo

Problem

There is a secret string S which consists of four characters 'A', 'B', 'X', or 'Y'. The first character of S never reappears in S . You know only N , the length of S .

You can ask a query as follows:

- You specify a string of the above four characters whose length is not larger than $4N$.
- You get the maximum length of the prefixes of S which are also substrings of the specified string.

Your task is to determine S .

Subtasks and Solutions

Let Q be the number of asks.

$N=3$

- Ask by all possible strings until N is returned.

$Q=4N$

- Determine the characters of S from the beginning one by one.
- For each position, try by all four characters one by one.

$Q=2N+1$ or $2N$

- Determine the characters of S from the beginning one by one.
- For the first position, determine the character by three asks. For each position except the first one, considering the constraint of S , determine the character by two asks.
- Or for each position, using conditional branches, determine the character by two asks.

$Q=N+2$

- Determine the characters of S from the beginning one by one.
- Except the first or last position, determine the character by one ask as follows.
 - Assume that the first character of S is, for example, 'A'.
 - Let s be the prefix of S already known.
 - Ask by $s + 'B' + s + 'XB' + s + 'XX' + s + 'XY'$.
 - If the next character is 'B', then $|s| + 1$ is returned. If it is 'X', then $|s| + 2$ is returned. If it is 'Y', then $|s|$ is returned.
- Note that if $Q \leq N + 10$, you can get nearly full points according as Q .



Review of Seats

Problem

The numbers between 0 and $HW - 1$, inclusive, are written on a rectangle divided to $H \times W$ squares such that each square contains one number and each number is written on exactly one square.

Deal with the following queries one by one:

- Swap the positions of two numbers.
- Count the number of t between 0 and $HW - 1$ such that the squares where the numbers between 0 and t are written form a rectangle.

Subtasks and Solutions

Subtask 1 (HW is very small)

Determine whether a set of squares forms a rectangle or not in $O(HW)$ time and deal with each query in $O(H^2W^2)$ time.

Subtask 2 (HW is small)

Let the square where the number i is written be (r_i, c_i) . The sufficient and necessary condition of $(r_0, c_0), \dots, (r_t, c_t)$ forming a rectangle is

$$(\max r_i - \min r_i + 1)(\max c_i - \min c_i + 1) = t + 1.$$

Here i moves between 0 and t .

Check if these conditions hold from $t = 0$ to $t = HW - 1$ inductively in $O(HW)$ time.

Subtask 3 ($H + W$ is small)

If $(\max r_i - \min r_i + 1)(\max c_i - \min c_i + 1) > t + 1$, the next t satisfying the condition is not less than $(\max r_i - \min r_i + 1)(\max c_i - \min c_i + 1) - 1$.

So it is enough to determine this condition in $O(H + W)$ numbers.

This condition is determinable independently in $O(\log(HW))$ time using a segment

tree. Then each query is dealt with in $O((H + W) \log(HW))$ time.

Subtask 4 (Gaps of two swapped numbers are small)

Memorize max and min of r_i and c_i for each t and recalculate them for t between the two swapped numbers for each query.

Subtask 5 (One dimensional, namely, $H = 1$)

Take t arbitrarily and color the squares with numbers $0, \dots, t$ black and the squares with numbers $t + 1, \dots, W - 1$ white (and add white squares outside of the rectangle.)

The sufficient and necessary condition of black squares forming a rectangle is that the number of pairs of adjacent two squares with different colors is two.

Manage the numbers of such pairs of squares for all values of t and count the number of ts satisfying the condition efficiently using a lazy propagation segment tree and deal with each query in $O(\log W)$ time.

Subtask 6 (No additional constraints)

Like above, take t arbitrarily and color the squares with numbers $0, \dots, t$ black the squares with numbers $t + 1, \dots, HW - 1$ white (and add white squares outside of the rectangle.)

Pay attention to two-times-two squares (sets of adjacent four squares.)

The sufficient and necessary condition of black squares forming a rectangle is as follows:

- There are only four two-times-two squares which contain exactly one black square.
- There are no two-times-two squares which contain exactly three black squares.

Manage the numbers of such two-times-two squares for all values of t and count the number of ts satisfying the condition efficiently using a lazy propagation segment tree and deal with each query in $O(\log(HW))$ time.



Review of Werewolf

Problem

Given a connected undirected graph with N vertices and M edges. The vertices are numbered from 0 through $N - 1$.

Q queries are given. The query i ($0 \leq i \leq Q - 1$) is represented by four integers S_i, E_i, L_i, R_i satisfying $L_i \leq S_i$ and $E_i \leq R_i$. You want to travel from the vertex S_i to the vertex E_i . Your route must satisfy the following condition:

- Assume that you visit the vertices $V_0, V_1, V_2, \dots, V_p$ in this order ($V_0 = S_i, V_p = E_i$). Then there is an index q ($0 \leq q \leq p$) such that $L_i \leq V_0, V_1, \dots, V_q$ and $V_q, V_{q+1}, \dots, V_p \leq R_i$ are satisfied.

You start the travel in **human form**, transform yourself from **human form** to **wolf form** at the vertex V_q , and finish the travel in **wolf form**.

Your task is to determine whether it is possible to travel from the vertex S_i to the vertex E_i .

Subtasks and Solutions

Subtask 1 (7 points)

$$N \leq 100, M \leq 200, Q \leq 100$$

You choose a V vertex where you transform yourself from human form to wolf form.

For each choice of V , you need to decide whether it is possible to travel from S_i to V in human form (i.e. only using vertices whose indices are $\geq L_i$), and to decide whether it is possible to travel from V to E_i in wolf form (i.e. only using vertices whose indices are $\leq R_i$).

The time complexity of this solution is $O(QN(N + M))$.

Subtask 2 (8 points)

$$N \leq 3\,000, M \leq 6\,000, Q \leq 3\,000$$

Determine the set of vertices you can visit from S_i in human form, and determine the set of vertices you can visit from E_i in wolf form.

Then check whether these two sets intersect.

The time complexity of this solution is $O(Q(N+M))$.

Subtask 3 (34 points)

The cities are located on a line. In other words, $M = N - 1$ and no city is directly connected to more than 2 cities.

Let U_i be the set of the vertices which are reachable from S_i by passing only vertices with index at least L_i . Similarly, let V_i be the set of the vertices which are reachable from E_i by passing only vertices with index at most R_i . Note that U_i forms a range on the line on which cities are located. This range can be efficiently computed using doubling or Segment tree. V_i can be similarly computed. Then, we can answer the query by checking whether these two ranges intersect.

Subtask 4 (51 points)

No additional constraints

We can construct a rooted tree so that U_i forms a subtree. This can be done using adding vertices to a disjoint set union structure in the descending order of indices. Then, using Euler-Tour on this tree, we can obtain a sequence of vertices and every U_i corresponds to a contiguous segment of this sequence. We can compute similar sequence for V_i . Then, we can answer the query by checking whether two segments for U_i and V_i shares a vertex. This can be done by the sweep line algorithm with a segment tree. The time complexity of this solution is $O((Q + M) \log N)$.



Review of Mechanical Doll

Problem

The problem is to create a circuit of devices.

Devices have one or two exits each. In a circuit, devices are connected: each exit is connected to another (or possibly the same) device. The connections are decided by the contestants.

A ball is placed on one of the devices and travels through the circuit indefinitely. At each step, the ball leaves the present device through one of its exits and enters the connected device.

There are three types of devices: the origin, triggers, and switches.

The origin is where the ball starts.

Triggers are numbered from 1 to M . Contestants are given an array A of length N . The ball must visit triggers $A_0, A_1, \dots, A_{N-1}, A_0, A_1, \dots, A_{N-1}, A_0, A_1, \dots$ in this order.

Switches have two exits each. The two exits work alternately. Contestants can decide the number of switches.

The task is to decide the connections so that the circuit satisfies the following conditions.

- The ball returns to the origin after a finite number of steps.
- By the time the ball first returns to the origin, every switch has been visited even times.
- The ball first returns to the origin after entering triggers exactly N times. The consecutive serial numbers of these triggers are $A_0, A_1, \dots, A_{N-1}$.

The score is determined according to the number of switches. Contestants get the full score when $N + \log_2 N$ switches or less are used.

Subtasks and Solutions

Subtask 1 (2 points)

Each trigger appears in A at most once. $N + \log_2 N$ switches can be used.

- Connect triggers without switches.

Subtask 2 (4 points)

Each trigger appears in A at most twice. $N + \log_2 N$ switches can be used.

- Put a switch after each trigger which appears twice.

Subtask 3 (10 points)

Each trigger appears in A at most 4 times. $N + \log_2 N$ switches can be used.

- For each trigger which appears more than twice, use 3 switches to branch its exit into 4 exits (Imagine a balanced binary tree). For each trigger which appears 3 times, kill one out of the first 3 exits by connecting it to the "root" switch.

Subtask 4 (10 points)

$N = 16$. 20 switches can be used.

- Gather exits of all the triggers into one, and branch it into 16 exits with 15 ($= 2^4 - 1$) switches.

Subtask 5 (18 points)

There is only one trigger.

- (Solution A) ($2N$ switches, half score) Use $2^k - 1$ switches for k such that $2N > 2^k \geq N$, and kill excessive exits by connecting them to the "root" switch.
- ($N + \log_2 N$ switches, full score) In addition to Solution A, skillfully choose which exits to kill so that they are closely placed in the circuit. Then many switches are now unnecessary because their exits are both connected to the "root" switch.

Subtask 6 (56 points)

No additional constraints.

- ($2N$ switches, half score) Do Solution A for every trigger.
- (Solution B) ($2N$ switches, half score) Gather exits of all the triggers into one, and branch it using Solution A.
- ($N + \log_2 N$ switches, full score) In addition to Solution B, save switches by skillfully choosing which exits to kill.



Review of Highway Tolls

Problem

We are given an undirected and unweighted graph G with N vertices and M edges, and constants $1 \leq A < B$.

Two vertices s and t are fixed but they are unknown to us.

We want to find s and t by calling the following function fewer times:

- For each edge in G , you arbitrarily assign the weight of A or B to turn G into a weighted graph. Then, the function returns the length of a shortest path between s and t on (weighted) G .

Subtask and Solutions

- Overall constraints: $N \leq 90,000$, $M \leq 130,000$

Subtask 1 (5 points)

at most 100 function calls, G is a tree, $N \leq 100$, s is known

- Test every possible t .

Subtask 2 (7 points)

at most 60 function calls, G is a tree, s is known

- Sort the vertices by the distance from s . Then t can be found using binary search

Subtask 3 (6 points)

at most 60 function calls, G is a path

- Binary search

Subtask 4 (33 points)

at most 60 function calls, G is a tree

- One function call with weight of every edge A to find the distance between s and t in unweighted G .
- An edge e on the shortest path between s and t can be found using binary search.
- After removing e , the graph will be separated into two subtrees. Then you can perform the solution of Subtask 2 twice to find s and t separately.
- Centroid decomposition is possible but implementation will be tough.

Subtask 5 (18 points)

at most 52 function calls, $A = 1, B = 2$

- Let S be a subset of V , where V is the set of vertices in G .
- We set the weight of edges between S and $V \setminus S$ to 1. The weights of other edges are set to 2. Then, we can tell whether exactly one of s and t belongs to S by looking at the parity of the answer to the call.
- Thus we can compute $s \text{ xor } t$. Using this, we can also find s and t themselves.

Subtask 6 (21 + 10 points)

at most 52 or 50 function calls (21 or 31 points, respectively)

- Solution A: 21 points
 - A vertex v on a shortest path between s and t can be found using binary search.
 - Construct a BFS tree with root v . Then, we can use binary search again to find one of s and t .
 - The other can be found similarly.
- Solution B: 31 points
 - Find an edge e on a shortest path between s and t as in Subtask 4.
 - Let $e = uv$. Without loss of generality, we can assume s, u, v and t appears in this order on this shortest path.
 - Then we can prove that s is strictly closer to u than to v . Similarly, t is closer to v than to u .
 - Thus we have disjoint candidate sets S and T such that s and t are contained in S and T , respectively. At the same time, we can construct BFS trees of vertex sets S and T with roots u and v , respectively. We can suppose a shortest path goes only through e and edges in the BFS trees.
 - Now we can find s and t as in the last part of Subtask 4.



Review of Meetings

Problem

There are N mountains, numbered from 0 through $N - 1$ from left to right. The height of the mountain i is H_i ($0 \leq i \leq N - 1$). Exactly one person lives on each mountain.

You are going to hold Q meetings, numbered from 0 through $Q - 1$. To the meeting j ($0 \leq j \leq Q - 1$), you will invite all people living on the mountains between the mountain L_j and the mountain R_j , inclusive.

For each meeting, you can choose a mountain as the meeting place. If the mountain x is chosen as the meeting place, the cost of the meeting is calculated as follows:

- The cost of the meeting is the sum of the costs of all participants.
- The cost of the participant from the mountain y is the maximum height of mountains between the mountain x and the mountain y , inclusive. Particularly, the cost of the participant from the mountain x is H_x .

For each meeting, you want to find its minimum cost.

Constraints

- $1 \leq N \leq 750\,000$
- $1 \leq Q \leq 750\,000$
- $1 \leq H_i \leq 1\,000\,000\,000$ ($0 \leq i \leq N - 1$)
- $0 \leq L_j \leq R_j \leq N - 1$ ($0 \leq j \leq Q - 1$)
- $(L_j, R_j) \neq (L_k, R_k)$ ($0 \leq j < k \leq Q - 1$)

Subtasks and Solutions

Subtask 1 (4 points)

$$N \leq 3\,000, Q \leq 10$$

If a meeting place is given, you can calculate the cost of a meeting in $O(N)$. Thus, by testing every possible meeting place, the cost of a meeting can be calculated in $O(N^2)$ time.

The total time complexity is $O(N^2Q)$.

Subtask 2 (15 points)

$$N \leq 5\,000, Q \leq 5\,000$$

By iterating through the mountains with maintaining an upper envelope, you can get costs for all

meeting places in $O(N)$ time.

The total time complexity is $O(NQ)$.

Subtask 3 (17 points)

$N \leq 100\,000$, $Q \leq 100\,000$, $H_i \leq 2$ ($0 \leq i \leq N - 1$)

Find the longest contiguous subsequence consisting only of 1 by SegmentTree. The total time complexity is $O(N + Q \log N)$.

Subtask 4 (24 points)

$N \leq 100\,000$, $Q \leq 100\,000$, $H_i \leq 20$ ($0 \leq i \leq N - 1$)

For each meeting, divide the range at the heighest mountains and recursively solve the problem.

If you pre-calculate the answer to some ranges, such as maximal ranges in which heights of all mountains are at most some constant, and you prepare a proper data structure for Range Minimum Queries, you can get the minimum cost of a meeting in $O(\max\{H_i\} \log N)$ time.

Pre-calculation can be done in $O(\max\{H_i\}N)$ time.

The total time complexity is $O(\max\{H_i\}(N + Q \log N))$.

Subtask 5 (40 points)

No additional constraints.

For convenience, let's assume that all values of H are distinct (this does not matter much). For each meeting, we can assume that the index of the optimal meeting place is greater than or equal to $\operatorname{argmax}_{L \leq i \leq R}(H_i)$, because by reversing the array H and solving the same problem we can get a real answer.

We denote the problem of calculating the minimum cost of a meeting with the range $[L, R]$ as query $[L, R]$.

- Let $\operatorname{Cost}(L, R)$ be the answer to the query $[L, R]$.
- Let $\operatorname{RangeL}(v)$ be the smallest x such that $\operatorname{argmax}_{x \leq i \leq v}(H_i) = v$.
- Similarly, let $\operatorname{RangeR}(v)$ be the largest x such that $\operatorname{argmax}_{v \leq i \leq x}(H_i) = v$.
- Also let $S(v)$ be the array of length

$$\operatorname{RangeR}(v) - \operatorname{RangeL}(v) + 1$$

such that the i -th ($0 \leq i \leq \operatorname{RangeR}(v) - \operatorname{RangeL}(v)$) value of $S(v)$ is

$$\operatorname{Cost}(\operatorname{RangeL}(v), \operatorname{RangeL}(v) + i).$$

We are going to compute $S(v)$ for all v , and then it is easy to get answers to all queries. The order of indices in which we compute $S(v)$ is very important. Here, we use depth-first-search post-order of the cartesian tree of H .

We define the cartesian tree of H as the rooted tree such that lowest-common-ancestor of nodes u and v is the node $\operatorname{argmax}_{u \leq i \leq v}(H_i)$.

The cartesian tree can be obtained in linear time by an iteration with a stack data structure. It can be easily seen that every node of the cartesian tree has at most two children, one to the left and another to the right. Let $lc(v)$ be the left child of the node v and $rc(v)$ be the right child of the node v (here we assume that the node v has two children).

Now the remaining task is to somehow merge $S(lc(v))$ and $S(rc(v))$ into $S(v)$. Clearly, first some elements of $S(v)$ is exactly $S(lc(v))$.

All we need is to compute $\text{Cost}(\text{RangeL}(v), p)$, for all p ($v \leq p$).

Since H_v is the maximum value in the range $[\text{RangeL}(v), \text{RangeR}(v)]$, you can see

$$\text{Cost}(\text{RangeL}(v), p) = \min\{\text{Cost}(\text{RangeL}(v), v) + (p - v) \times H_v, (v - \text{RangeL}(v) + 1) \times H_v + \text{Cost}(v + 1, p)\}$$

and

$$\begin{aligned} & (\text{Cost}(\text{RangeL}(v), v) + (p - v) \times H_v) - ((v - \text{RangeL}(v) + 1) \times H_v + \text{Cost}(v + 1, p)) \\ \leq & (\text{Cost}(\text{RangeL}(v), v) + (p + 1 - v) \times H_v) - ((v - \text{RangeL}(v) + 1) \times H_v + \text{Cost}(v + 1, p + 1)) \end{aligned}$$

where $p + 1 \leq \text{RangeR}(v)$.

The inequality follows from the observation that

$$\text{Cost}(v + 1, p + 1) - \text{Cost}(v + 1, p) \leq \max_{v+1 \leq i \leq p+1} (H_i) \leq H_v.$$

It indicates that there exists a certain index z such that

$$\text{Cost}(\text{RangeL}(v), p) = \text{Cost}(\text{RangeL}(v), v) + (p - v) \times H_v$$

for all $p \leq z$, and

$$\text{Cost}(\text{RangeL}(v), p) = (v - \text{RangeL}(v) + 1) \times H_v + \text{Cost}(v + 1, p)$$

for all $z < p$.

Therefore, you can get $S(v)$ in the following way:

- Let T be the array obtained by adding a certain value to all elements of $S(rc(v))$.
- Update first some elements of T with a certain linear function.
- Concatenate $S(lc(v))$, $\text{Cost}(\text{RangeL}(v), v)$, and T .

To carry out these operations fast, we use a compressed representation for $S(v)$. $S(v)$ is represented by the list of ranges. Each range has a certain linear function such that the values of $S(v)$ in the range can be calculated by the linear function.

Adding a certain value can be done by lazy propagation.

To update first some elements, we simply iterate through T from the beginning. The ranges before the break point is replaced by one range, so the total number of iterations is $O(N)$.

Concatenating two arrays can be done as follows:

- We maintain end points of ranges in a global set and store the information of ranges in a global array. Then, we do not have to do anything for ranges.

- Concatenating lazy propagation information for adding can be done by Weighted-union heuristic:
 - Let $W(s)$ be the value which should be added to elements of the array s .
 - When we concatenate two arrays s and t , we pick the smaller one and arrange the elements of it so that $W(s) = W(t)$.
 - By Weighted-union heuristic this can be done in $O(N \log N)$ time in total.

Getting the answer to a query requires one lower bound operation of the set. Thus in $O(Q \log N)$ time we can get answers to all queries.

The total time complexity is $O((N + Q) \log N)$.